

# FactoryMethod

Um arquiteto está mobiliando uma casa pelo software que um programador criou. Ao analisar o código, você percebe que a instanciação dos objetos são totalmente concretas, e totalmente acopladas ao Cliente. Aplique o pattern FactoryMethod e explique como você aplicou abaixo:

## Resumo do Projeto: Implementação do Padrão Factory Method

Este projeto demonstra a aplicação do padrão de design Factory Method para a criação de diferentes tipos de mobília ( `IMobilia` ). O objetivo é fornecer uma estrutura flexível e extensível para a criação de objetos de mobília sem acoplar o código cliente às classes concretas.

## Estrutura do Projeto

### 1. Interfaces e Entidades

- `IMobilia` : Interface que define as propriedades e métodos comuns a todas as mobílias.
- `IMobiliaFactory` : Interface que define o método `CreateMobilia` para criar objetos `IMobilia` .

### 2. Fábricas Concretas

- `SofaFactory` : Implementa `IMobiliaFactory` para criar objetos `Sofa` .
- `TableFactory` : Implementa `IMobiliaFactory` para criar objetos `Table` .
- `ChairFactory` : Implementa `IMobiliaFactory` para criar objetos `Chair` .

### 3. Factory Creator

- `MobiliaFactoryCreator` : Classe que centraliza a criação das fábricas de mobília. Contém um dicionário que mapeia strings para instâncias de `IMobiliaFactory` e fornece o método `GetMobiliaFactoryByString` para obter a fábrica correta com base em uma string.

### 4. Modelo Principal

- `MainModel` : Classe que utiliza `MobiliaFactoryCreator` para converter uma lista de strings em uma lista de objetos `IMobilia` e enviar essa lista para um

arquivo de texto.

## Fluxo de Trabalho

1. **Criação das Fábricas:** `MobiliaFactoryCreator` instancia e armazena as fábricas concretas ( `SofaFactory` , `TableFactory` , `ChairFactory` ) em um dicionário.
2. **Conversão de Strings para Objetos:** `MainModel` utiliza `MobiliaFactoryCreator` para obter a fábrica correta com base em uma string e criar instâncias de `IMobilia` .
3. **Envio para Arquivo:** `MainModel` escreve a lista de objetos `IMobilia` em um arquivo de texto, incluindo detalhes como nome, preço, material e cor.

## Exemplo de Código

- **Interface** `IMobiliaFactory` :

```
public interface IMobiliaFactory
{
    IMobilia CreateMobilia();
}
```

- **Fábrica Concreta** `SofaFactory` :

```
public class SofaFactory : IMobiliaFactory
{
    public IMobilia CreateMobilia() ⇒ new Sofa();
}
```

- **Classe** `MobiliaFactoryCreator` :

```
public class MobiliaFactoryCreator : IMobiliaFactoryCreator
{
    private readonly Dictionary<string, IMobiliaFactory> _mobilia;

    public MobiliaFactoryCreator()
    {
        _mobilia = new()
        {
            {"Sofa", new SofaFactory()},
        }
    }
}
```

```

        {"Table", new TableFactory()},
        {"Chair", new ChairFactory()}
    };
}

public IMobiliaFactory GetMobiliaFactoryByString(string mobilia)
    ⇒ _mobilia.ContainsKey(mobilia) ? _mobilia[mobilia] : throw new Ar
gumentException("Mobilia not found");
}

```

- **Classe** **MainModel** :

```

public class MainModel(IMobiliaFactoryCreator mobiliaFactory) : IMain
Model
{
    private readonly IMobiliaFactoryCreator _mobiliaFactoryCreator = mo
biliasFactory;

    public List<IMobilia> ConvertMob(List<string> mobList)
    {
        var convertedMobList = new List<IMobilia>();

        foreach (var mobilia in mobList)
        {
            if (!string.IsNullOrEmpty(mobilia))
            {
                IMobiliaFactory mobiliaFactory = _mobiliaFactoryCreator.Get
MobiliaFactoryByString(mobilia);
                IMobilia convertedMobilia = mobiliaFactory.CreateMobilia();

                convertedMobList.Add(convertedMobilia);
            }
        }
        return convertedMobList;
    }

    public Task SendMob(List<IMobilia> mobList)
    {

```

```

        using StreamWriter streamWriter = new StreamWriter("Mobilia.txt");

        streamWriter.WriteLine("Mobilia List");
        streamWriter.WriteLine("-----");

        foreach (var mobilia in mobList)
        {
            streamWriter.WriteLine("Mobilia: " + mobilia.Name);
            streamWriter.WriteLine("Price: " + mobilia.Price);
            streamWriter.WriteLine("Material: " + mobilia.Material);
            streamWriter.WriteLine("Color: " + mobilia.Color);
            streamWriter.WriteLine("-----");
        }

        return Task.CompletedTask;
    }
}

```

## Conclusão

O projeto demonstra como o padrão Factory Method pode ser utilizado para criar uma estrutura flexível e extensível para a criação de objetos. A centralização da lógica de criação em `MobiliaFactoryCreator` simplifica o código cliente e facilita a adição de novas fábricas de mobilia.